Presentation
On
Implementation Details
of

RepoLens

AI-Assisted RepositoryIntelligenceforSecurity& Code Understanding

Project implementation, architecture, workflows, APIs and evaluation plan

Group Number: 05

Presented by

Rounak Kumar Choudary • Rajat Negi • Priyanka Singh • Priyanshi Bora

Department of CSE
GNIOT (Greater Noida, India)

Under the Supervision
Mr. Inmar Ansari

(Approved by AICTE, Delhi & Affiliated to Dr. A.P.J. Abdul Kalam Technical University, Lucknow)

Plot No. 7, Knowledge Park-II, Greater Noida, Gautam Buddh Nagar, Uttar Pradesh-201310

Index

DOCUMENT MAP

01 **Project overview & scope**
Purpose, design goals, boundaries

02 **System architecture**
Components and request/data flow

03 **Repository structure**
Key folders, modules and responsibilities

04 **Runtime workflows**
Repository mode and paste mode

05 **Workspace state**
State ownership and result caching

06 **Local analysis engine**
Supported files, rule families and severity

07 **File intelligence**
AST metadata and progressive call-flow graph

08 **Structured outputs**
AnalysisResult, RepoSummary and compact AI payload

09 **API implementation**
Scan, file, context and explain endpoints

10 **AI integration & trust boundaries**
What AI does today and what it does not do

11 **Security, privacy & operations**
Secrets, input validation and deployment notes

12 **Testing & evaluation**
Manual checks, metrics and test strategy

13 **Limitations & roadmap**
Known gaps and prioritized work

14 **Setup & references**
Run commands and technical sources

1. Project overview & scope

What RepoLens / Scrutiny does

A local-first source-code review tool. It discovers repository paths, fetches a source file only when selected, runs deterministic local rules and extracts lightweight structure. A user may then request a Gemini-generated summary of local results.

What it is not

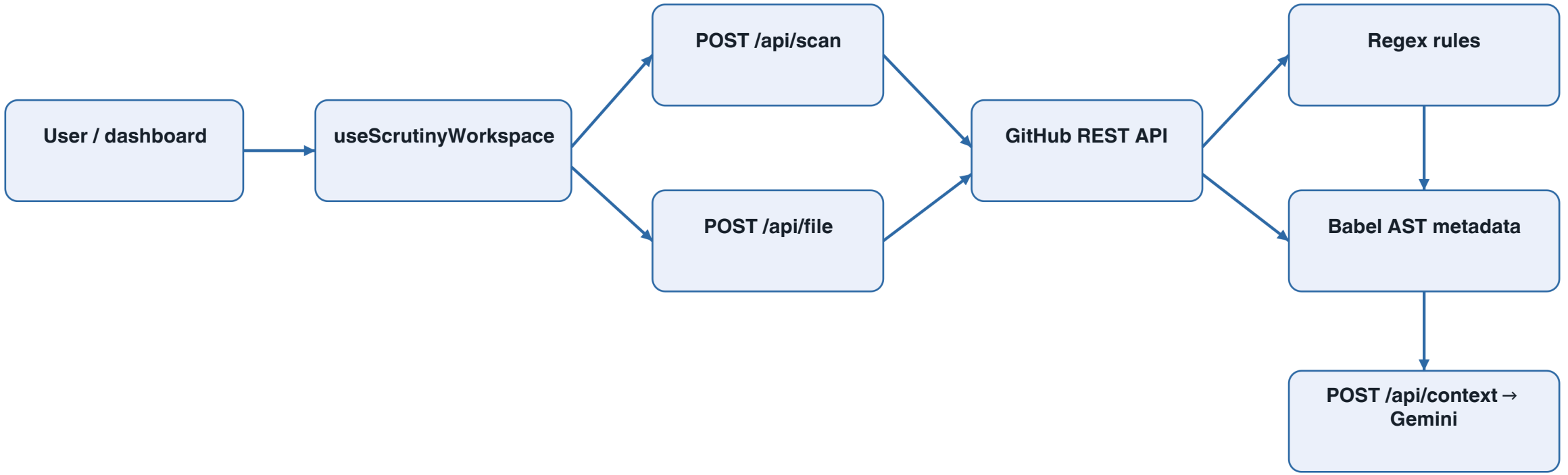
It is not a complete security scanner and does not prove exploitability. It may miss real vulnerabilities or flag safe code. Findings are review signals; a developer or security reviewer must confirm each result.

Core design goals

- Run predictable checks without requiring an AI call.
- Fetch only the file the user selects; do not download every source file during discovery.
- Keep findings and metadata in browser memory for the current page session.
- Send compact analysis data to Gemini only after the user chooses Generate Insights.
- Support paste mode without requiring a GitHub repository.
- Show inspectable evidence: file path, line, matched text and nearby code.

2. System architecture

HIGH-LEVEL DATA FLOW

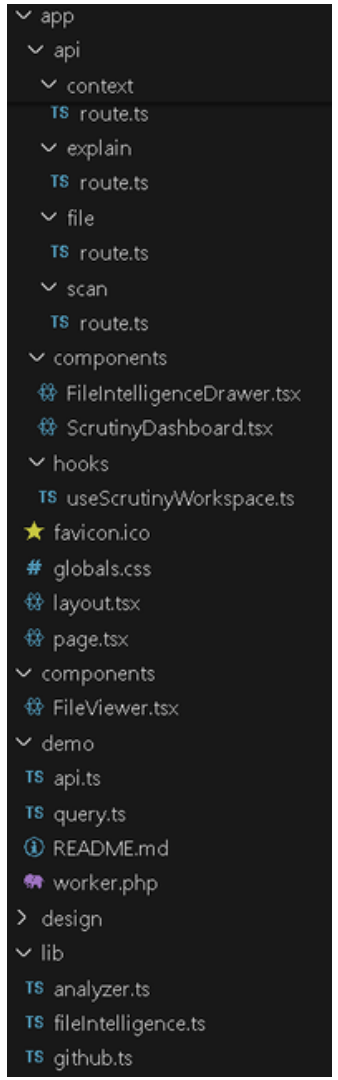


Local analysis produces findings and metadata. AI is an optional explanation/summary step, not the current vulnerability detector.

Important: repository scan returns paths; selected-file requests fetch source; the user explicitly triggers Gemini insights.

3. Repository structure

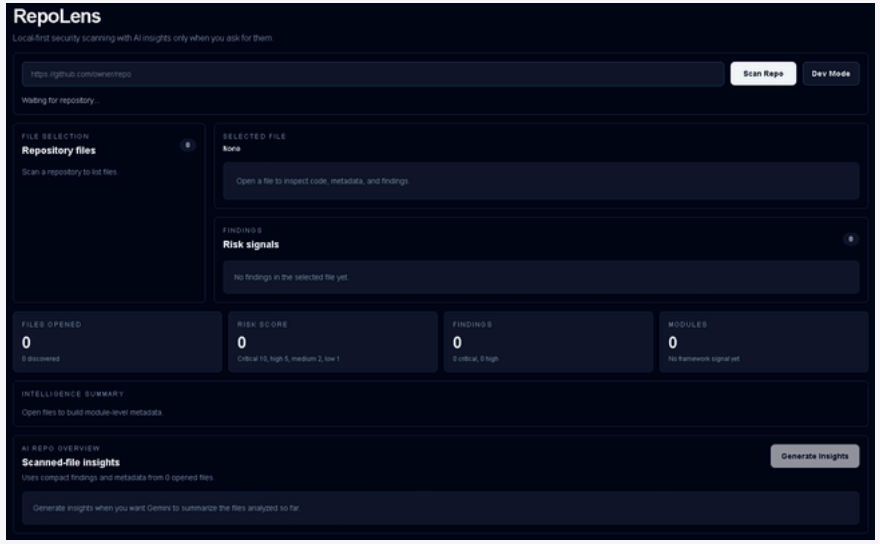
`app/page.tsx`	Creates the workspace hook and passes state to the dashboard.
`app/hooks/useScrutinyWorkspace.ts`	Owns scan state, selected file, local analysis, paste mode and insight cache.
`app/components/ScrutinyDashboard.tsx`	Scan controls, file list, source view, findings, summaries, insights and drawer.
`app/components/FileIntelligenceDrawer.tsx`	Selected-file AST details and progressive cross-file call-flow graph.
`app/api/scan/route.ts`	Validates a GitHub repository URL and returns discovered file paths.
`app/api/file/route.ts`	Validates repository URL + path, then returns one source file.
`app/api/context/route.ts`	Sends compact local-analysis summary to Gemini and returns Markdown.
`app/api/explain/route.ts`	Separate Gemini endpoint; current dashboard does not call it.
`lib/github.ts`	Parses GitHub URLs and calls GitHub REST API.
`lib/analyzer.ts`	Security rules, JS/TS parsing and repository summaries.
`lib/fileIntelligence.ts`	AST details and static HTTP source-file call links.
`demo/`	Four-file TypeScript/PHP fixture for local rules and insights.



4. Runtime workflows

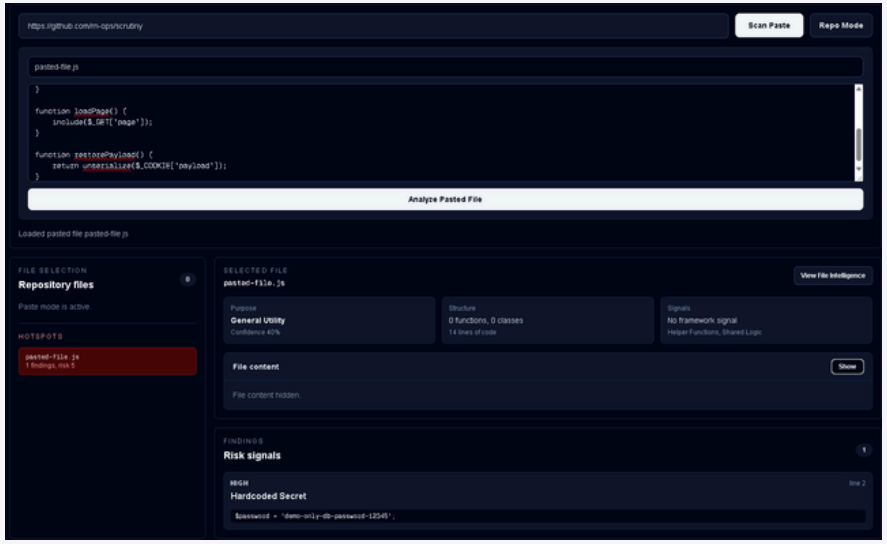
Repository mode

1. User enters a GitHub repository URL.
2. POST /api/scan validates it and returns paths.
3. User selects a file; POST /api/file fetches that file.
4. Local analyzer creates findings and metadata.
5. Workspace stores results by path and refreshes the UI/graph.
6. Generate Insights sends a compact summary to /api/context.



Paste mode

1. User pastes a path and source text.
2.. Browser calls analyzeFiles directly.
3. Local rules and metadata appear without GitHub API access.
4. User may separately request AI insights.
5. The same compact-summary flow sends analysis output to /api/context.



Repository totals have a specific meaning

The fileCount value describes discovered paths. Most other totals describe files that have actually been opened and analyzed in the current session. Opening a file adds or replaces its FileIntelligence record; the graph grows progressively.

5. Workspace state

Repository	repoUrl, files
Selected file	selectedFile, fileContent
Analysis	analysis: accumulated findings, metadata and repo summary
File intelligence	fileIntelligenceByPath: AST-derived calls, imports, functions and references
Finding selection	selectedFinding
Paste mode	devMode, devCode, devFilePath
AI insight state	repoInsights, insightsLoading, insightsError, insightsCache

Cache caveat The AI cache is in-memory and clears on reload. Its current short integer hash can collide; production should use a stable cryptographic digest or store and compare the full normalized payload.

6. Local analysis engine

Input filtering
Skips empty paths/content, unsupported extensions and configured ignored path segments.

Line scanning
Tests extension-specific rules line by line; skips common comment-prefix lines and lines containing scan-ignore.

Evidence
A finding records ID, path, line, severity, rule title, matched text, code snippet, nearby context and a generated security story.

Supported extensions

JavaScript, JSX, TypeScript, TSX, Python, PHP, C, C++, C/C++ headers, Java and C#. Babel AST metadata is strongest for JavaScript/TypeScript-family files; other languages mainly receive regex-based rules.

Current rule families

- All supported files: hardcoded secret assignments.
- JavaScript / TypeScript: process execution, shell use, eval, unsafe HTML insertion, document.write and prototype keys.
- Python / PHP: process execution, unsafe deserialization or loading, dynamic evaluation, SQL interpolation and path-related patterns.
- C / C++: unsafe copy functions, non-literal format strings and simple allocation-size multiplication patterns.
- Java / C#: process execution, path construction and SQL command construction/concatenation.

7. Severity, AST & file intelligence

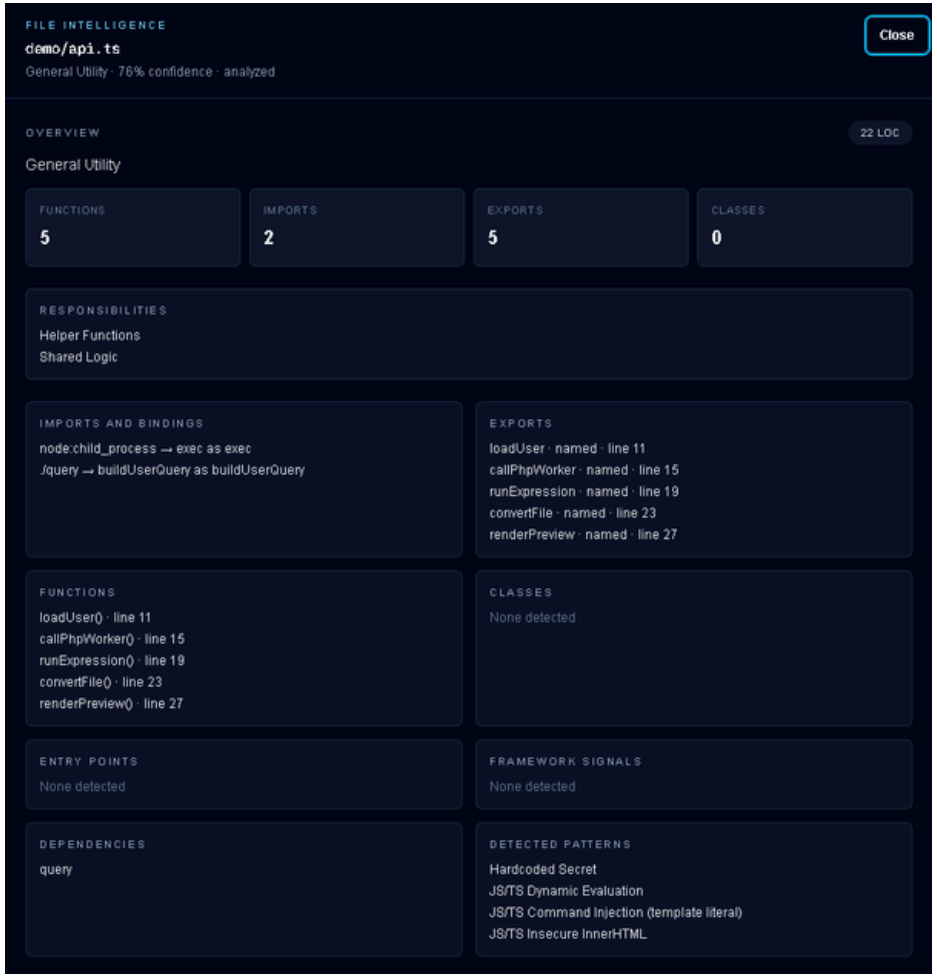
Risk score weights

File and repository risk scores are sums of finding weights. They are sorting signals, not calibrated probabilities, CVSS scores or estimates of exploit likelihood. False positives can contribute to the score.

CRITICAL · 10	MEDIUM · 2
HIGH · 5	LOW · 1

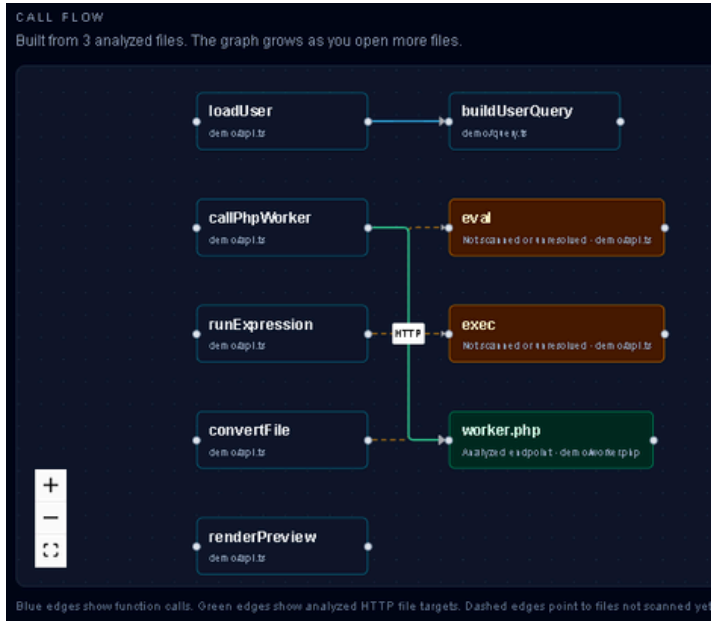
AST metadata and graph

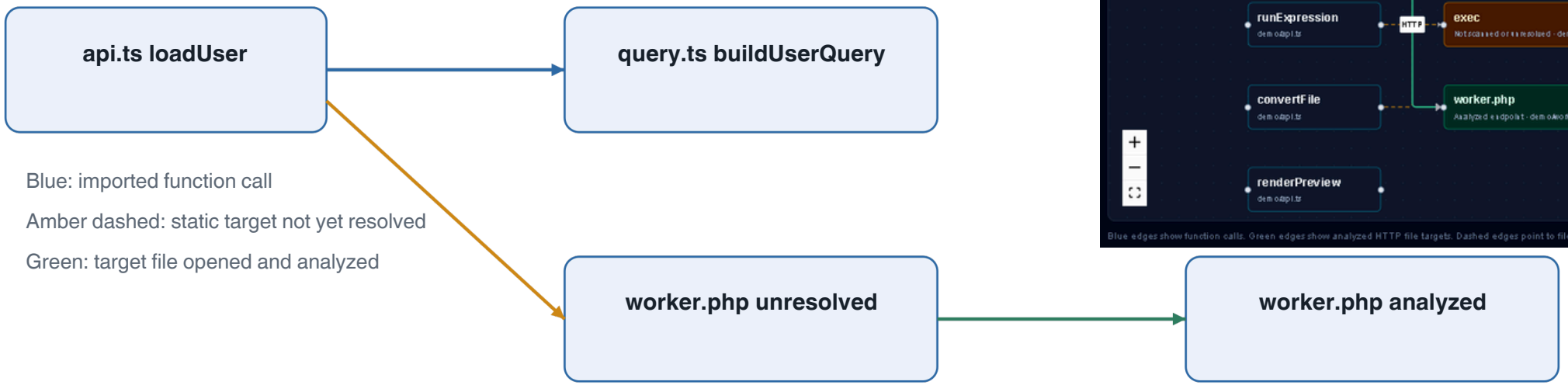
- For JS/TS, Babel collects imports, exports, functions, classes, call edges, framework hints, entry points and dependencies.
- Module role labels and basic source metrics are heuristics, not guaranteed semantic facts.
- The graph combines function calls within parsed files and static fetch() links to supported source-file extensions.
- Imported-function links resolve only when both files have been analyzed and the target function is exported.
- Blue edges show function calls; green edges show analyzed HTTP targets; dashed amber edges mark unresolved or unanalyzed targets.



8. Progressive call-flow graph

GRAPH GROWS FROM OPENED FILES





Static graph limitations

The graph does not prove that a route exists or runs. It cannot resolve dynamic URL construction, arbitrary HTTP clients, process execution, reflection or runtime dispatch. PHP and Python files end at file nodes because Babel does not parse their function-level AST.

9. Structured results

AnalysisResult

Groups three outputs:

- findings — local rule matches
- metadata — per-file structure and inferred role
- repoSummary — totals, modules, hotspots, frameworks and dependencies

RepoSummary

Groups analyzed files by inferred purpose, totals severity/risk and ranks up to five file hotspots. Values change as more files are opened.

CompactAiPayload

Limits the AI prompt: counts and hotspots, up to 8 module summaries, 25 analyzed files and 8 findings per included file. Omits full source and most finding context.



Why compact the payload?

The limits reduce prompt size and avoid sending full source files through the current insight flow. They can also omit lower-ranked files or findings, so the AI summary is not a complete review of every repository path.

10. API implementation

REQUEST AND RESPONSE RESPONSIBILITIES

`POST /api/scan` Input: repository URL	Validates GitHub URL; helper reads repository metadata/default branch and recursive tree; returns success + paths or an error.
`POST /api/file` Input: repository URL + path	Validates both fields; fetches one file via GitHub Contents API; expects Base64 content and decodes UTF-8.
`POST /api/context` Input: URL, mode, analysis object	Serializes local analysis into a Gemini prompt; returns Markdown. It does not validate AI claims against source.
`POST /api/explain` Input: one finding and code context	Separate Gemini explanation route. Current dashboard does not call it; fixed local explanations are used for known rule types.

11. AI integration & trust boundaries

CURRENT BEHAVIOR VS FUTURE OPTION

Current implementation

Local rules create findings. Gemini is called only after Generate Insights, and /api/context summarizes compact local results. This is AI-assisted explanation of existing results, not AI-based vulnerability discovery.

Possible future second pass

A future AI detection mode should return separate candidate findings with strict schema and file/line evidence. Validate every path, line, severity and evidence against source; reject invalid output and never overwrite rule severity.

Trust boundary rules

- Model output is probabilistic and can invent paths or misread code; it is not proof.
- Do not place GitHub or Gemini secrets in browser code.
- Show consent before any future mode sends source snippets to a hosted model.
- Keep local findings distinct from AI candidates and require human verification.

12. Security, privacy & operations

DEPLOYMENT GUARDRAILS

- GitHub access uses server-side GITHUB_TOKEN when configured; it can increase API rate limits and may permit private repository access.
- GEMINI_API_KEY is required for Generate Insights. Keep both credentials private in .env.local or deployment secret settings.
- Validate every route input at runtime. TypeScript types alone do not validate untrusted JSON request bodies.
- Treat repository code and model output as untrusted input; sanitize Markdown according to the renderer in use.
- A public service should add rate limits, request-size limits, access controls and abuse monitoring.
- Current insights requests send compact metadata and selected finding evidence, not complete source files.
- Use explicit user action for hosted AI requests; keep local analysis useful without an AI key.

Recommended release gate

Do not describe results as verified vulnerabilities until each result has code-backed evidence and the evaluation demonstrates useful precision and recall.

13. Testing & evaluation

MEASURE QUALITY, NOT ALERT VOLUME

Test dataset

Label vulnerable and safe examples. Cover each supported language and rule family. Vary names, formatting and safe fixes; keep some projects out of development data.

Compare modes

Use the same labeled set for rule-only baseline, AI-only review and hybrid review. Record model/version, prompt, temperature, token size and rule version.

Report metrics

Precision, recall, F1, false alerts per 1,000 LOC, detection by language/family, cost per repository, latency, failure rate and reviewer confirmation time.

Minimum manual checks before demo

- Use the four-file demo fixture to check known safe and unsafe patterns.
- Verify path validation and error responses for malformed repository URLs and paths.
- Open multiple files and check that graph nodes/edges update without duplicate stale records.
- Test missing GitHub token, missing Gemini key, API errors and empty/unsupported files.
- Confirm the UI labels scores as rankings and presents evidence for human review.

14. Limitations & roadmap

PRIORITY IMPLEMENTATION WORK

P0 · Correctness	Add unit tests for every rule with safe and unsafe examples; add regression tests for overlapping findings.
P1 · Evidence quality	Measure precision/recall before expanding coverage; improve source-to-sink and AST-aware analysis where feasible.
P1 · API hardening	Add runtime schemas, request limits, model-cost telemetry and abuse controls.
P2 · Graph confidence	Test cross-file imports, HTTP links, unresolved targets and recursive call graphs.
P2 · AI second pass	Optional structured AI candidates with strict evidence validation, consent and separate UI presentation.

15. Setup & technical references

Local setup

Requirements: Node.js and npm
1. npm install
2. npm run dev
3. Open http://localhost:3000
Optional .env.local:
GITHUB_TOKEN=… and GEMINI_API_KEY=…
Scripts: npm run lint · npm run build · npm run start

Operational note

The current package scripts do not define a dedicated automated test suite. Use demo/ for manual checks and add labeled tests before trusting detection metrics. Keep credentials out of source control.

Primary project source

Scrutiny Project Documentation — github.com/rn-ops/scrutiny/design/PROJECT_DOCUMENTATION.md

Technical references cited by the project documentation

- NIST SP 800-218, Secure Software Development Framework (SSDF), Version 1.1 (2022).
- GitHub CodeQL documentation — code databases, syntax, data-flow and control-flow context.
- Gao et al., “How Far Have We Gone in Vulnerability Detection Using Large Language Models” (arXiv:2311.12420, 2023).
- Yu et al., “A Preliminary Study of Large Language Models for Multilingual Vulnerability Detection” (arXiv:2505.07376, 2025).